CS4432: Database Systems II

Indexing

Basics

Updates

- Mon **Apr 6**: **Quiz 3**
  Remember to bring your laptops

- **Project 1** Deadline: **Apr 3 (tomorrow)**

- **HW2 is posted**
  - **Due: Apr 9**

Update to the tentative schedule

Week	Date	Day	Topic	Assignment	Notes
1	16-Mar-26	MON	**Intro, Logistics,** and Disks and Storage		
1	19-Mar-26	THU	Disks and Storage	HW1 Posted	
2	23-Mar-26	MON	Record Representation	Q1	
2	26-Mar-26	THU	Buffer Management	HW1 Due & P1 Posted	
3	31-Mar-26	**TUE (MON)**	External Sorting	Q2	**Follow Monday sche**
3	2-Apr-26	THU	Indexing	P1 Due & HW2 Posted	
4	6-Apr-26	MON	B-Tree Index	Q3	
4	9-Apr-26	THU	B-Tree Index	HW2 Due & P2 Posted	Reflection?
5	13-Apr-26	MON	Hash Indexing		
5	16-Apr-26	THU	Relational Algebra Recap	Q4 & P2 Due	
~~6~~	~~20-Apr-26~~	~~MON~~			~~Patriots Day~~
6	23-Apr-26	THU	Query Processing	HW3 Posted & P3 Posted	
7	27-Apr-26	MON	Query Processing		
7	30-Apr-26	THU	**Final Exam**		
8	4-May-26	MON	Recap and extended topics	HW3 Due & P3 Due	

Quiz 2

100%	40%	81%	80%	00:11:10
High Score	Low Score	Mean Score	Median Score	Mean Elapsed Time

Quiz 2

Question

True Or False

A record with "*n*" variable-length fields will have "*n*" offset slots (one for each variable field) in the record header.

n-1

0.46
Item Difficulty

0.46/1
PTS
Mean Earned Points

0/1 PTS
Median Earned Points

0.53
Discrimination Index

0.21
Corrected Item-Total Correlation Coefficient

Answer Frequency Summary

	Answer	Respondents	Percentage
✕	True	29	54%
✓	False	25	46%

Quiz 2

Question

Multiple Choice

In the buffer manager, assume the buffer pool consists of 5 frames, which are all initially empty. The replacement policy is LRU (Least Recently Used). "Read P_i" and "Update P_i" represent requests to read or update the content of disk page Pi, respectively. Assume that pages are immediately copied to the disk after an 'Update'. The following sequence of disk page requests will result in how many disk I/Os in total?

Read P1, Read P2, Update P1, Read P3, Read P8, Read P7, Read P6, Read P2

0.65
Item Difficulty

0.65/1 PTS
Mean Earned Points

1/1 PTS
Median Earned Points

0.57
Discrimination Index

0.27
Corrected Item-Total Correlation Coefficient

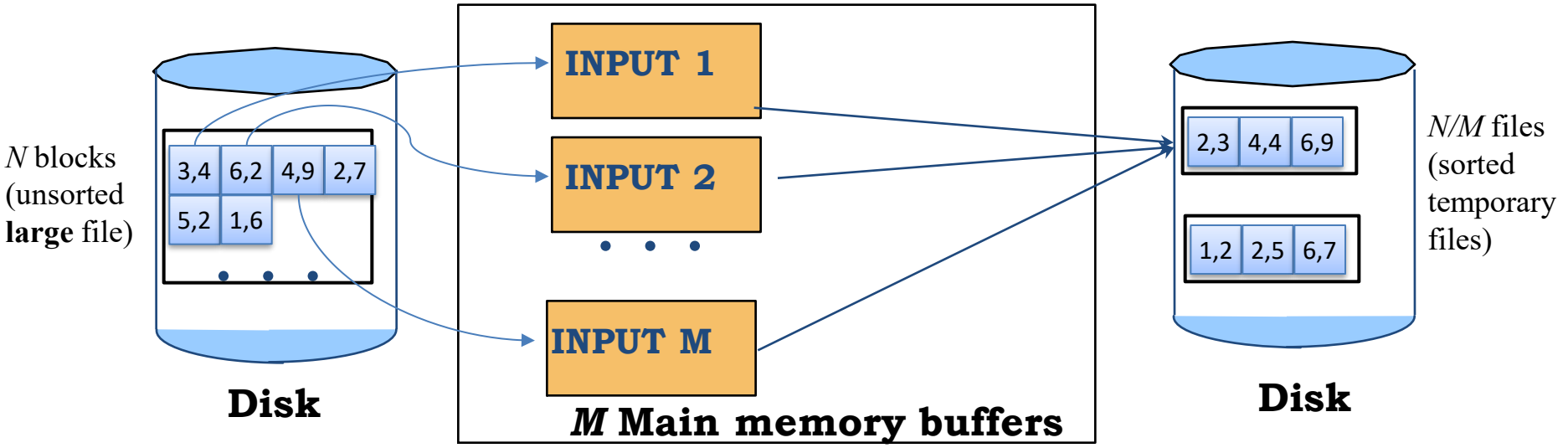
Answer Frequency Summary

	Answer	Respondents	Percentage
×	6	5	9%
×	7	10	19%
✓	8	35	65%
×	9	4	7%

Read P1, Read P2, Update P1, Read P3, Read P8, Read P7, Read P6, Read P2

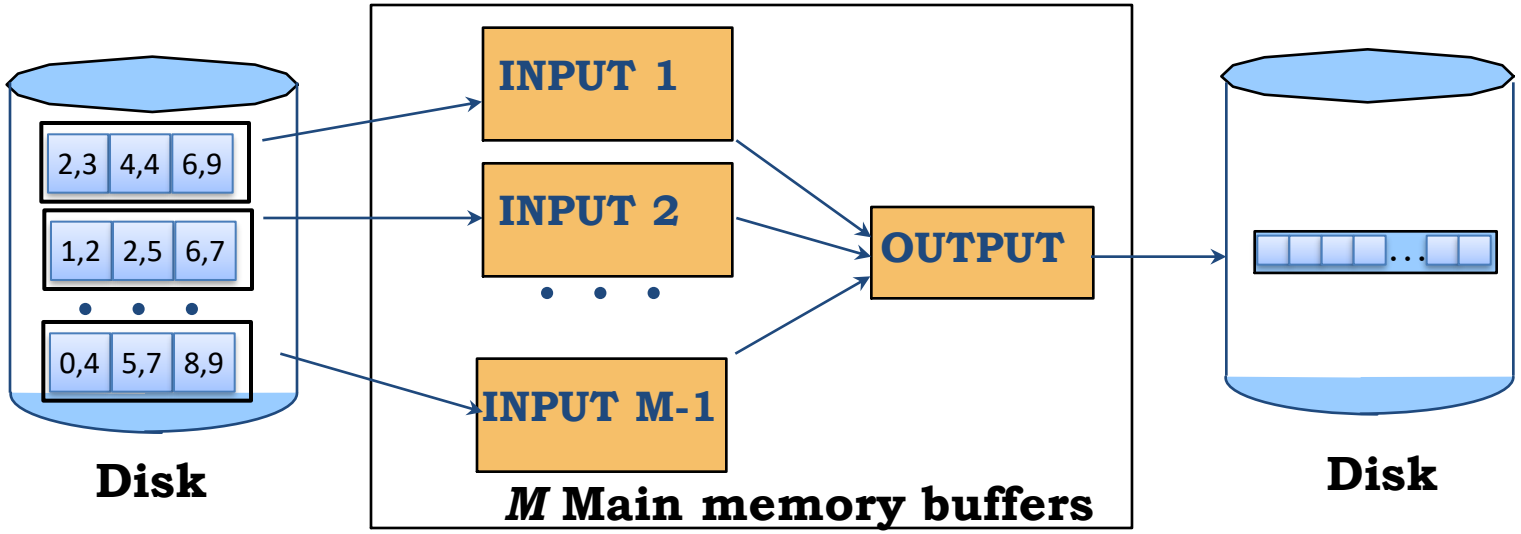
Step	Operation	Hit/Miss	Dirty?	Buffer (MRU → LRU)	I/O This Step	Cumulative I/O
1	Read P1	MISS	clean	**P1**	1 read	**1**
2	Read P2	MISS	clean	**P2, P1**	1 read	**2**
3	Update P1	HIT	**P1 becomes dirty**	**P1, P2**	0	**2**
4	Read P3	MISS	clean	*P3, P1, P2**	1 read	**3**
5	Read P8	MISS	clean	*P8, P3, P1, P2**	1 read	**4**
6	Read P7	MISS	clean	*P7, P8, P3, P1, P2**	1 read	**5**
7	Read P6	MISS → evict **P2** (clean)	clean	**P6, P7, P8, P3, P1***	1 read	**6**
8	Read P2	MISS → evict **P1*** (dirty → write-back)	clean	**P2, P6, P7, P8, P3**	**1 read + 1 write = 2**	**8**

M-way External Merge Sort
Phase 1 : Prepare



- Construct as large as possible **(sorted)** starter lists.
- Sort M blocks at once (in-place algorithm)
  - Output: ONE SORTED LIST composed by M blocks
- ➔ Will reduce the number of needed passes

M-way External Merge Sort
Phases 2, 3, … : Merge



- **Merge as many sorted sublists into one long sorted list.**
- **Keep 1 buffer for the output**
- **Use M-1 buffers to read from M-1 lists**

Cost of External Merge Sort

- Number of passes: $1+\lceil \log_{M-1} \lceil N / M \rceil \rceil$
- Total I/O Cost = 2N * (# of passes)

Example

- Buffer : with M=5 buffer pages
- File to sort : N=108 pages
  - Pass 0:
    - Number of runs? Size of each run?
  - Pass 1:
    - Number of runs? Size of each run?
  - Pass 2:
    - Number of runs? Size of each run?
  - Pass 3:
    - Number of runs? Size of each run?

Example

- Buffer : with M=5 buffer pages
- File to sort : N=108 pages
  - Pass 0: [redacted] sorted runs of 5 pages each (last run is only 3 pages)
  - Pass 1: [redacted] sorted runs of 20 pages each (last run is only 8 pages)
  - Pass 2: 2 sorted runs, 80 pages and 28 pages
  - Pass 3: Sorted file of 108 pages

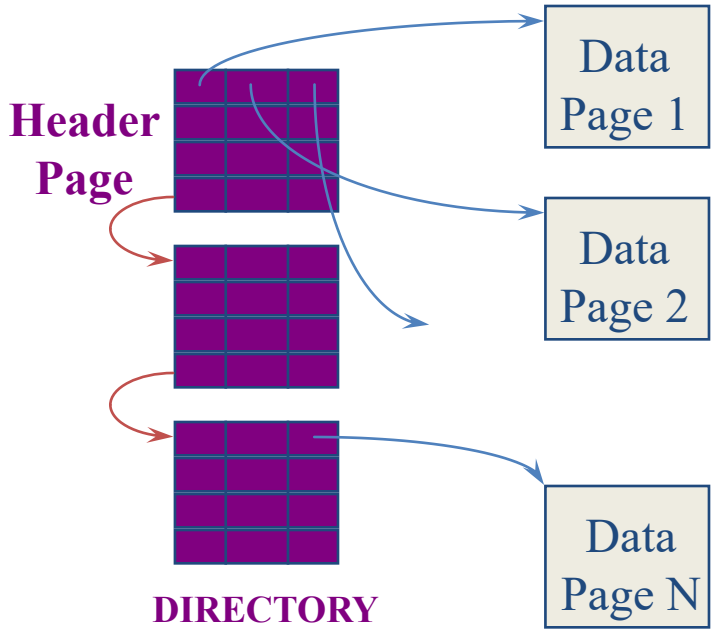
Learning objectives

- Explain how **indexes** are used in DBs, how they improve **performance** and when incur **overhead**
- Describe the **differences** between **Dense** and **Sparse** indexes regarding: (i) whether files need to be sorted, (ii) insertions, (iii) deletions
- Understand how a **secondary index** is created after a table is sorted based on primary index
- Explain how **multi-level** indexes are used for solving equality and range queries efficiently

Locating Records: Table Scans

```
Select ID, name, address
From R
Where ID = 1000;
```

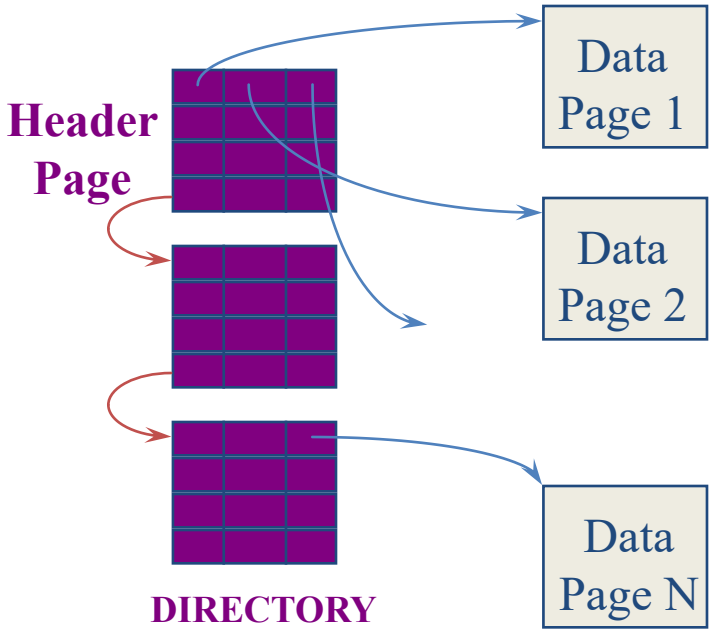
Knowing what we know by now,
how do we execute this query?



Naïve way ➔ Table Scan

- Open the **files** of relation R
- Access each data page
- Access each record in each page
- Check the condition

Locating Records: Table Scans

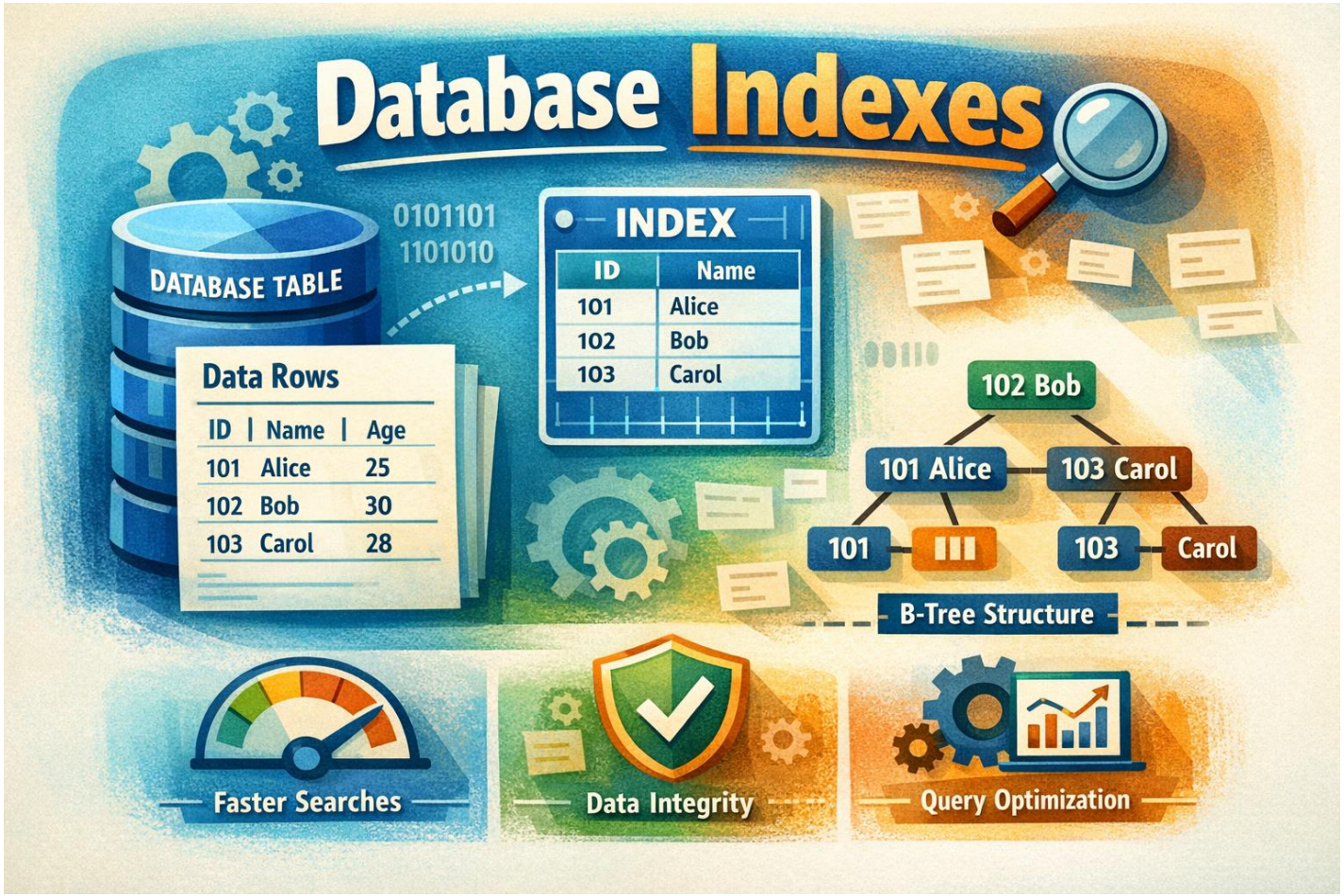


- Open the files of relation R
- Access each data page
- Access each record in each page
- Check the condition

What is the I/O cost? → **N**

What is the least amount of memory needed for Table Scan? → **Only 1 memory block**

HOW CAN WE USE SORTING TO ACCESS RECORDS MORE EFFICIENTLY?



Locating Records: Index Scan

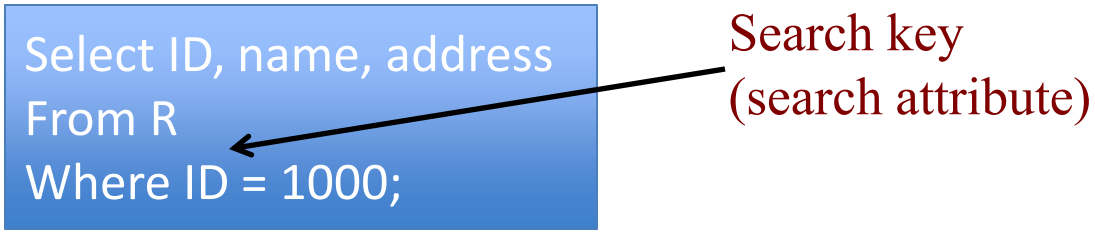
```
Select ID, name, address
From R
Where ID = 1000;
```

- Table Scan is always an option
- But it is not efficient, especially for large relations

- **➔ Index Scans are more efficient**
  - But require DBMS to index the table beforehand!

Basic Concepts

- Indexing mechanisms are used to **speed up** access to desired data.
- **Search Key** - attribute to set of attributes used to look up records in a file.



- An **index file** consists of records (called **index entries**) of the form

Can I create index for column that is NOT a primary key?

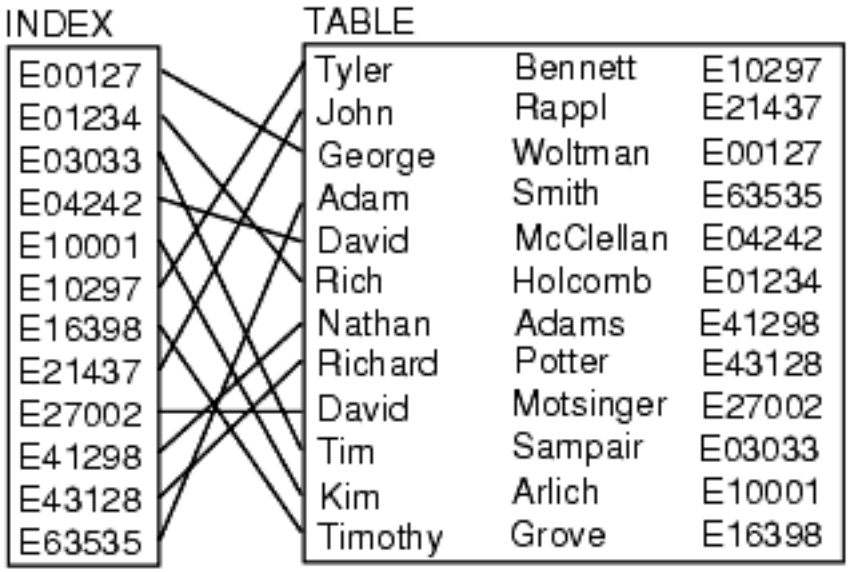
Yes, can create index for any column.

Basic Concepts (Cont'd)

- An **index file** consists of records (called **index entries**) of the form

- Index files are typically much smaller than the original file

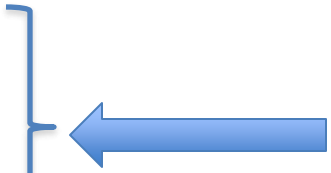
- **Types of indexes**
  - Dense vs. Sparse
  - Primary vs. Secondary
  - One-Level vs. Multi-Level



Index Evaluation Metrics

- Query time
- Insertion time
- Deletion/Update time
- Space overhead

Savings here



Overheads here

- **Access types supported. E.g.,**
  - **Equality Search** (x = 100): records with a specified value in the attribute
  - **Range Search** (10 < x < 100): records with an attribute value falling in a specified range of values.

Sequential Files & Primary Indexes

Primary index *is an index on the* ***ordering column***

10	
20	

30	
40	

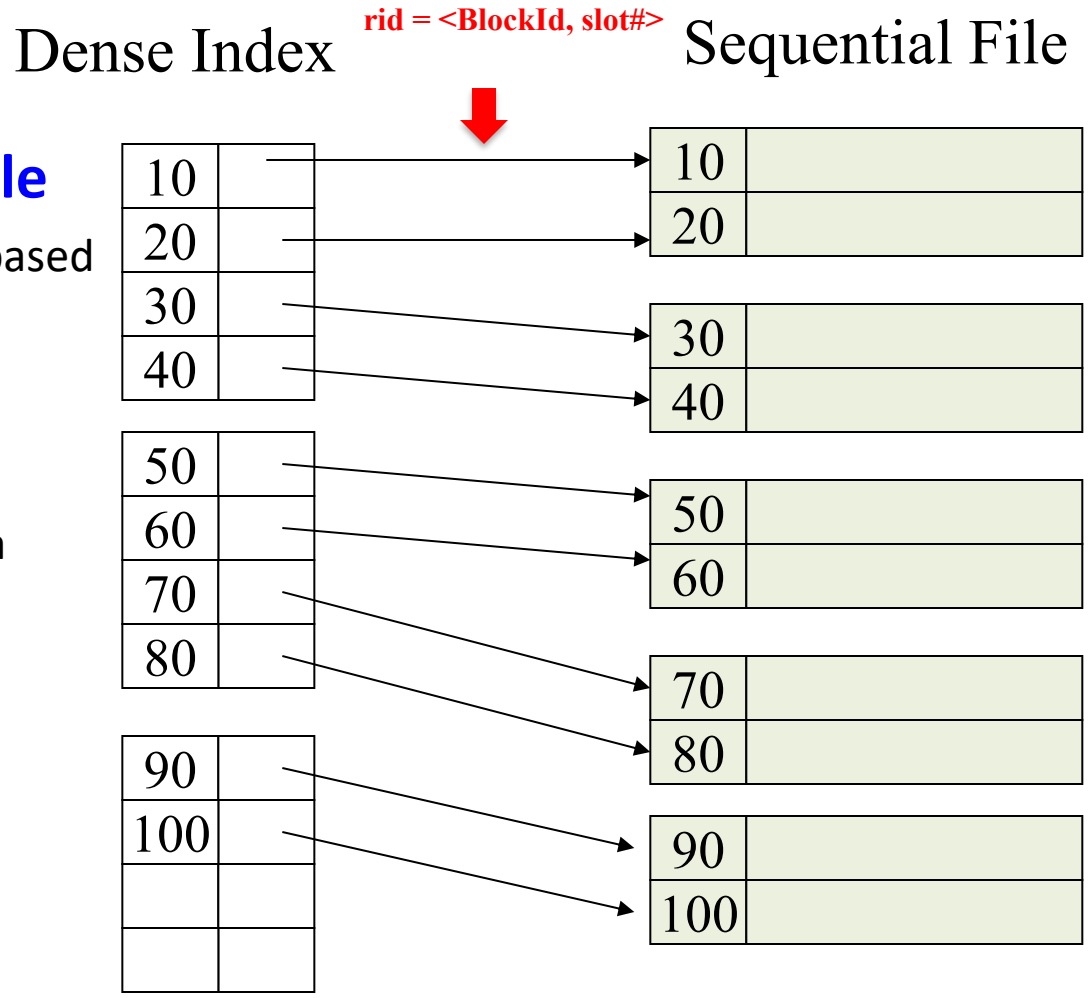
50	
60	

70	
80	

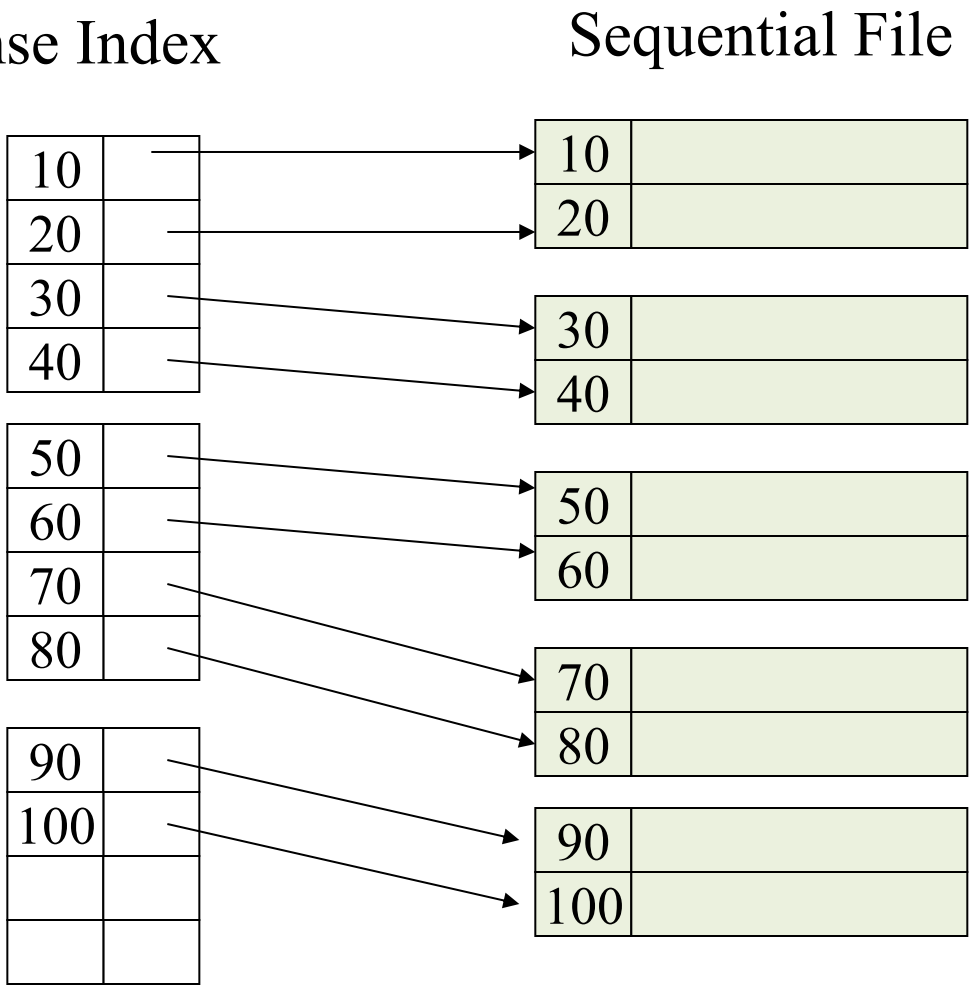
90	
100	

Dense Index on Ordered File

- **Ordered (Sequential) File**
  - Records are stored sorted based on the indexed attribute
- **Dense Index**
  - Has one entry for each data tuple



Dense Index on Ordered File



#entries in index = #records in file

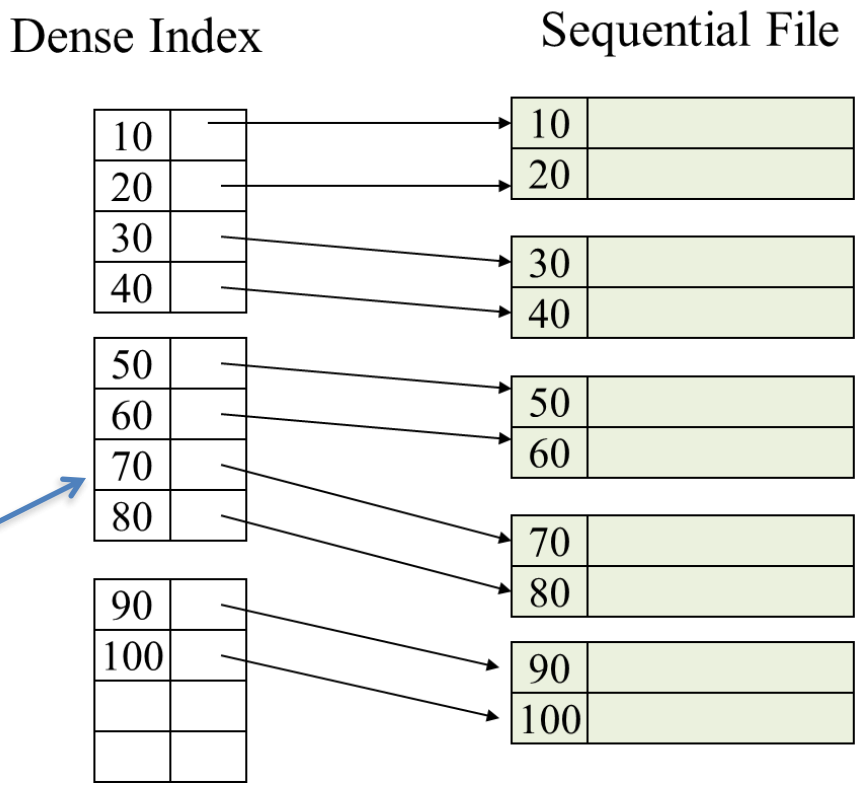
But the index size is much smaller than the file size

Can we build a dense index on unordered file?

Yes

Dense Index: Locate Key = 100

- **Index Scan**
  - Read each page from the index
  - Search for key = 100
  - Follow the ***pointer ➔ (Record Id)***
- **Index Binary Search**
  - Since all keys are sorted
  - Read middle page in index
  - Either you find the key, Or
    - Move up or down

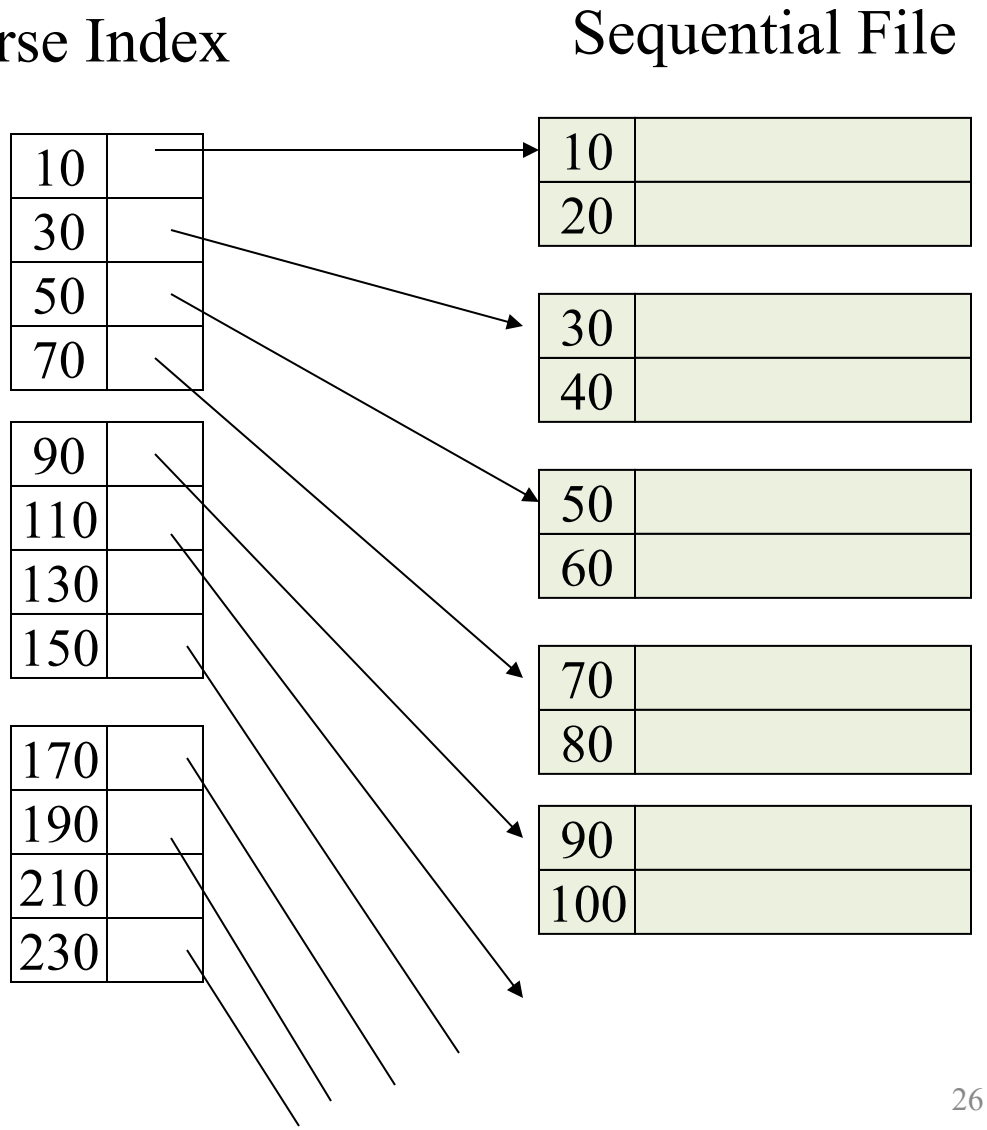



```
Select ID, name, address
From R
Where ID = 100;
```

Sparse Index On Ordered File

- **Sparse Index**
  - An entry for only the 1st record in each data block

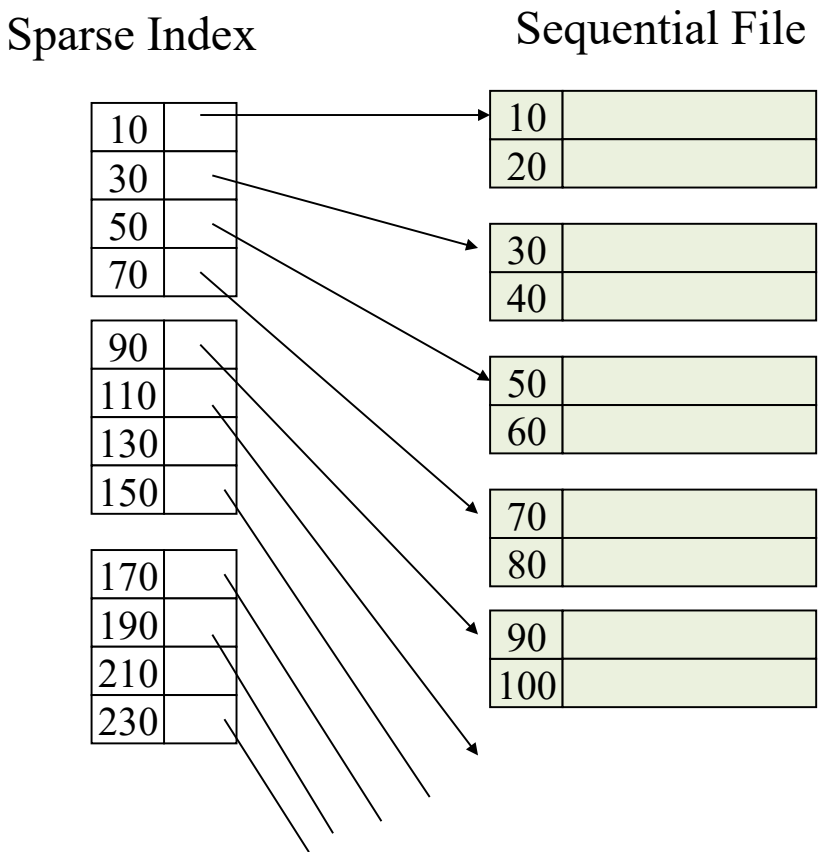
Sparse Index is smaller than a dense index



Sparse Index On Ordered File

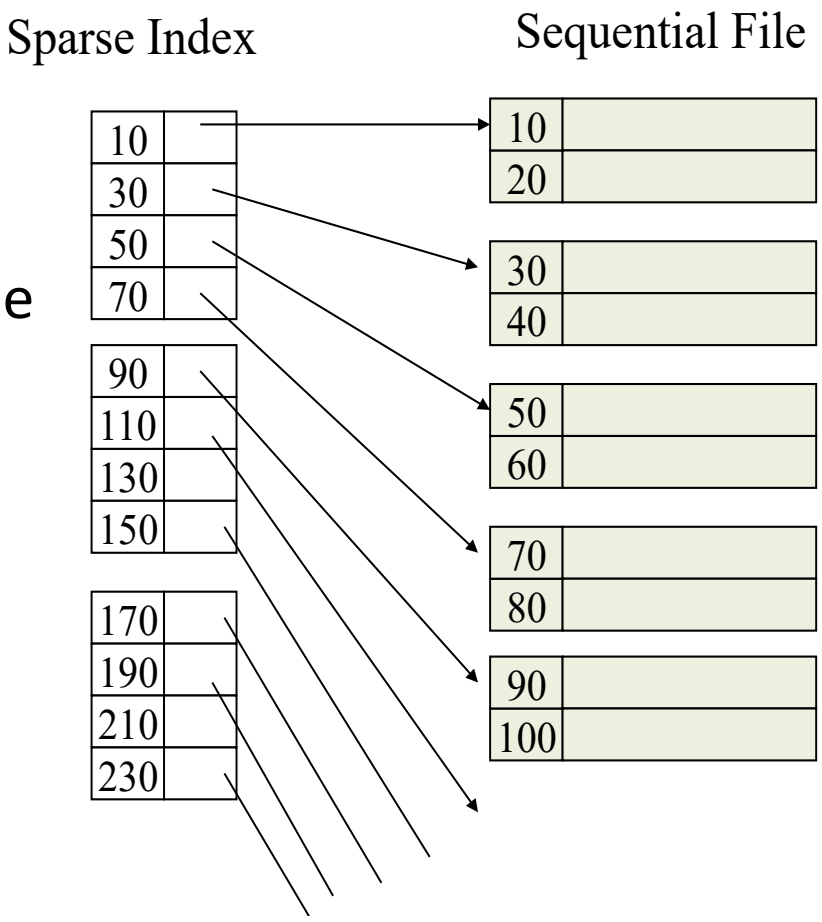
Can we build a sparse index on unordered file?

- **NO**
- **Sparse Indexes can be built ONLY on ordered (sequential files)**



Sparse Index: Locate Key = 100

- **Index Binary Search still works**

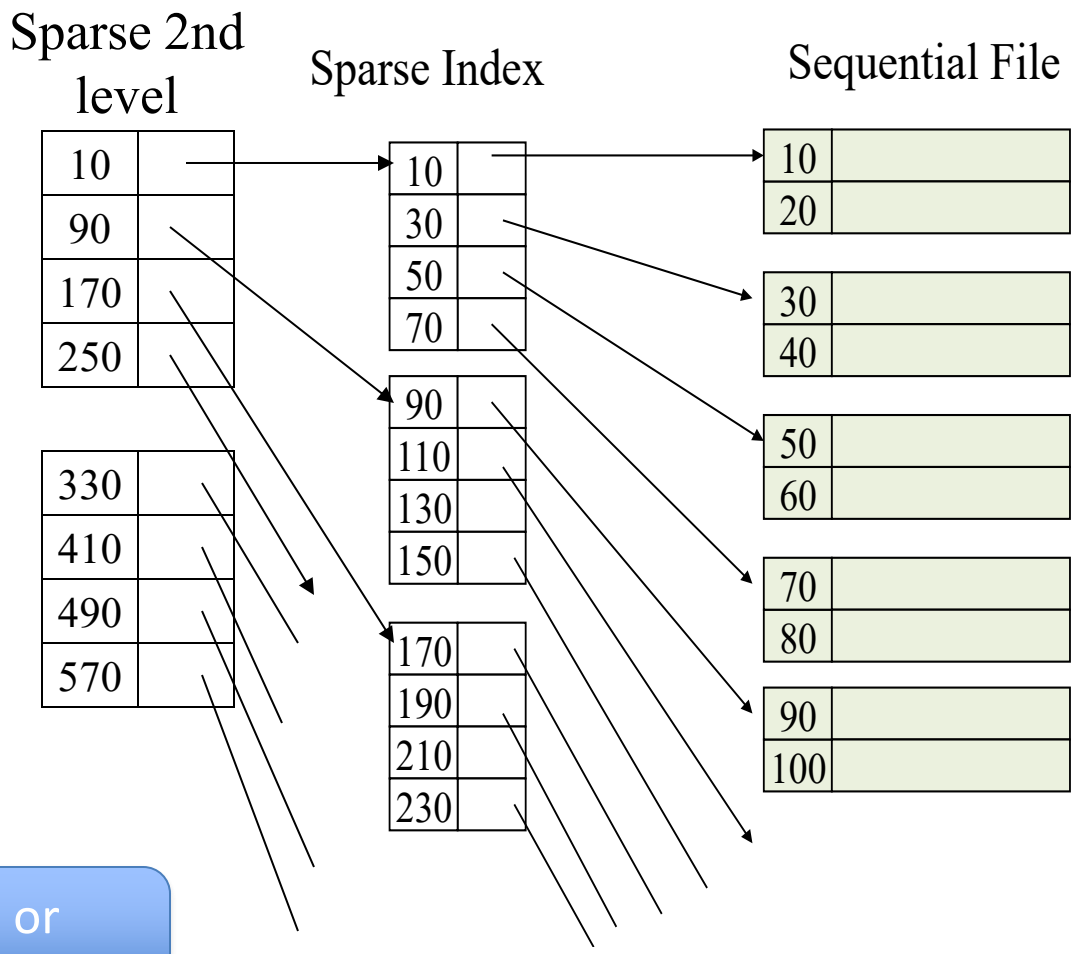


- Either locate the search key in the index, **OR**
- Locate the largest key smaller than your search key

- Follow the ***pointer and check the data block***

Multi-Level Index

- 1st level Index file is just a file with sorted keys
- We can build a 2nd level index on top of it



Is the index file always sorted?

Is the 2nd level sparse or dense? Can it be dense?

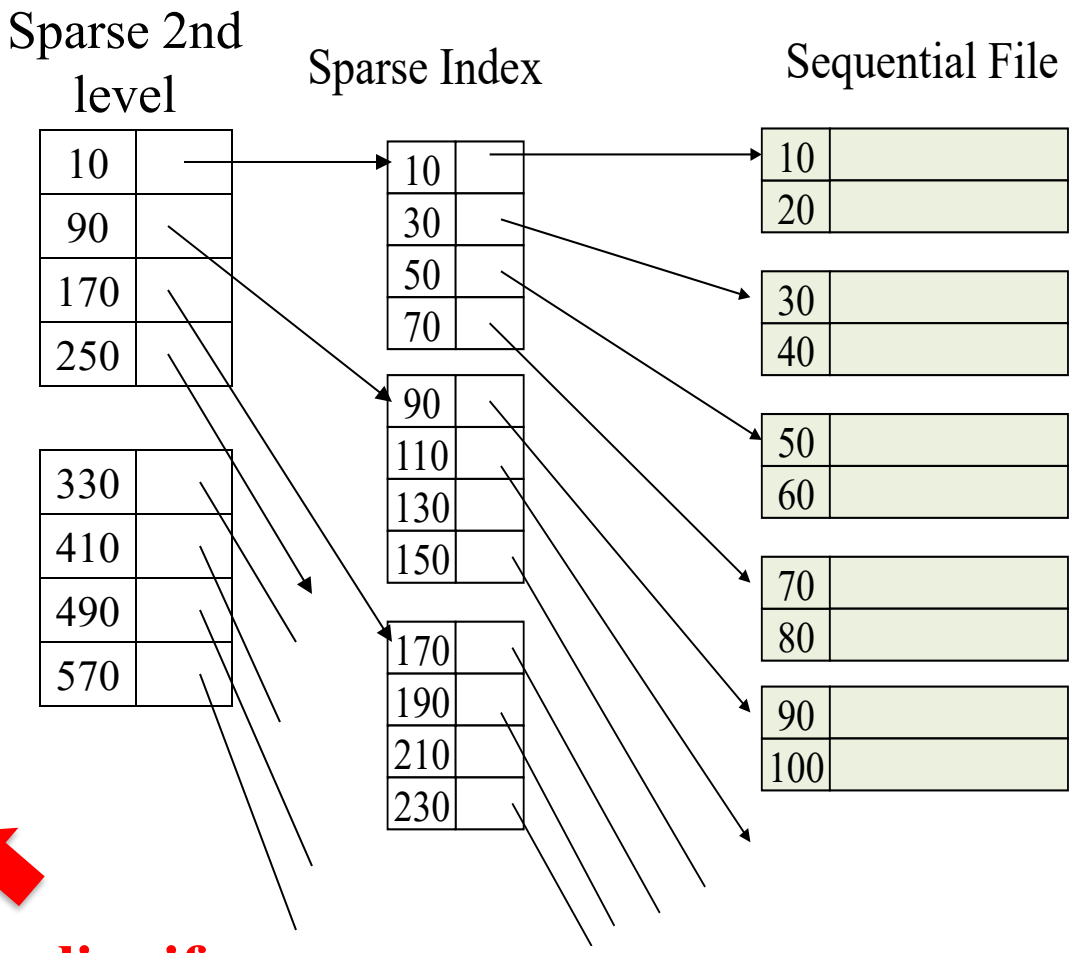
Multi-Level Index

Is the index file always sorted?

Yes

Is the 2nd level sparse or dense? Can it be dense?

2nd, 3rd, … levels have to be sparse (otherwise no savings)



Same idea applies if 1st level is dense i.e., higher levels must be sparse

Sparse vs. Dense Indexes

- **Dense**
  - More space
  - Must use for unsorted files (secondary indexes)
  - Can tell if record does not exist without checking the data file
- **Sparse**
  - Less space
  - Only for sorted files or higher-level indexes
  - Cannot tell if record does not exist without checking data file

Sparse vs. Dense Indexes Summary

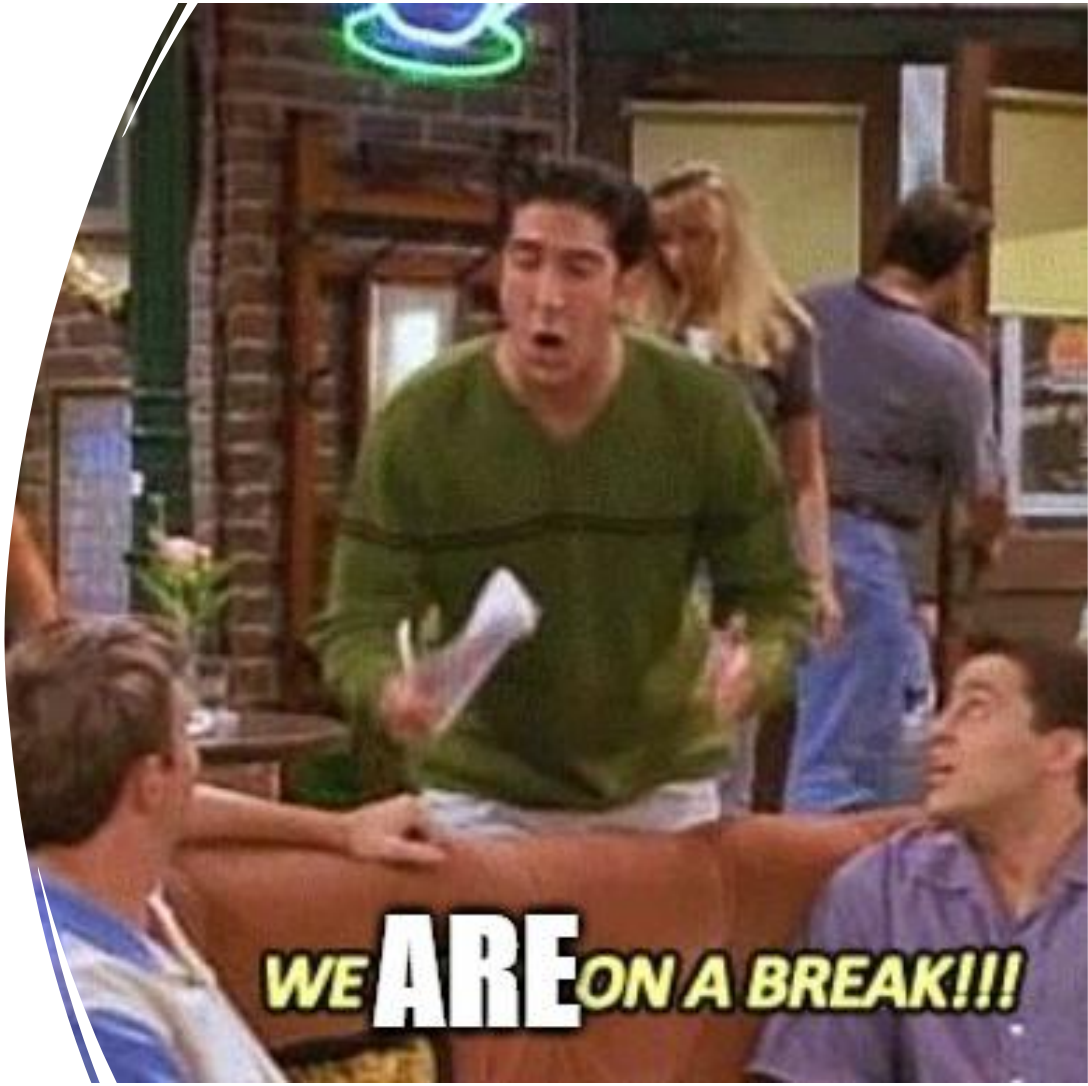
- **Benefits vs. Overheads of Indexes**
- **Sparse**
  - Less space
  - Better for insertion
  - Only for sorted files (or higher-level indexes)

- **Dense**
  - More space
  - Must use for unsorted files (secondary indexes)
  - Can tell if record does not exist without checking the data file

- Beyond the 1st level, all following levels are **sparse**
- Duplications adds some complications to searching algorithm

Break

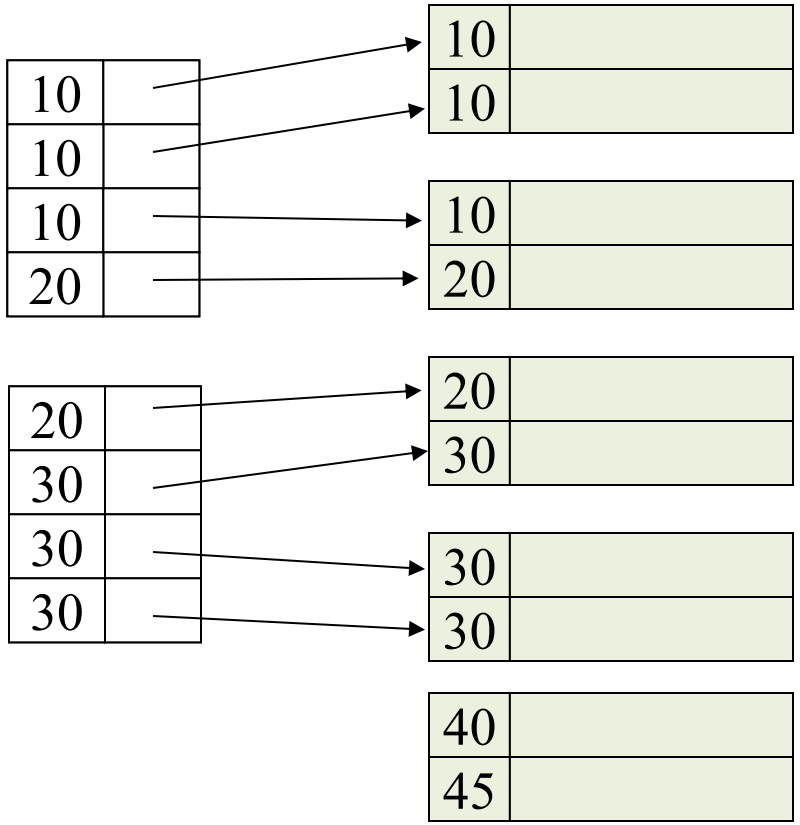
- Now we will take a few minutes break



Files with Duplicate Keys: Dense Index

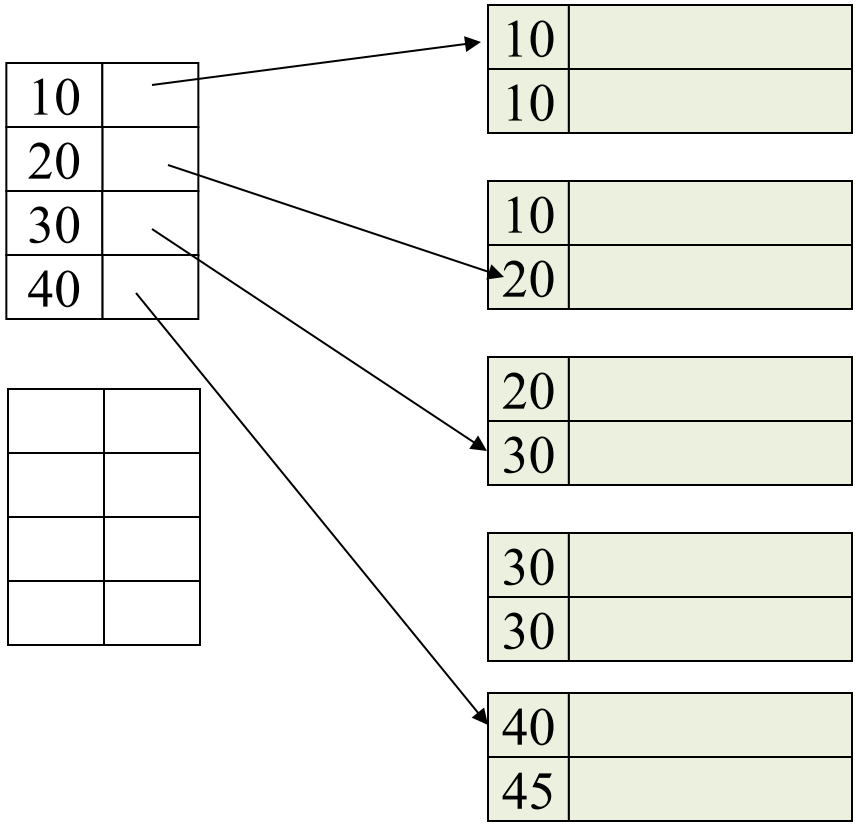
- Entry in the index for each value

Too much wasted space



Files with Duplicate Keys: Dense Index (Compact Design)

- Entry in the index for each distinct value



- **How to locate key 35**
  - It does not exist in the index and the index is dense
  - No need to search the data file

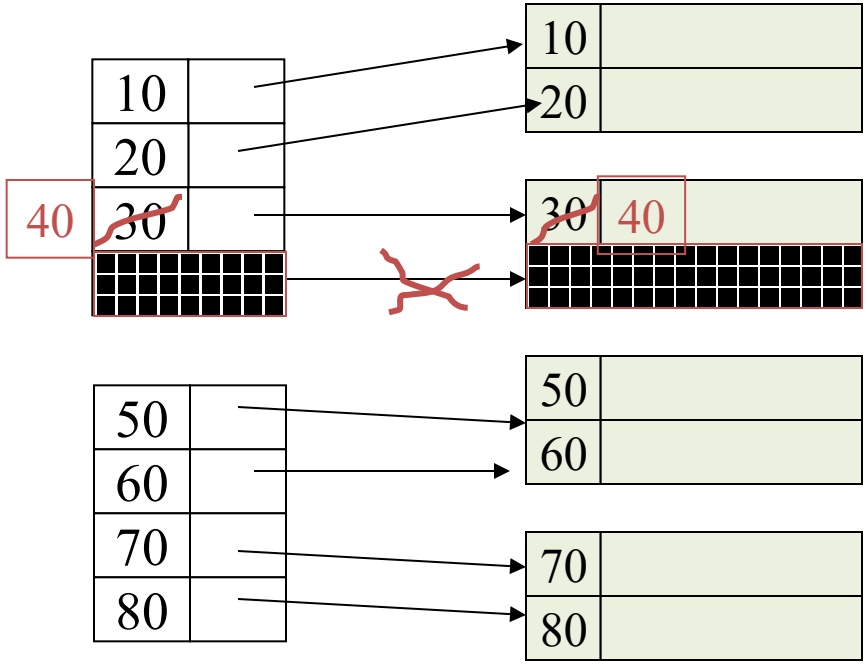
Sequential (Ordered) File

Insertion/Deletion
on
Primary Indexes

Dense Index: Deletion

– Delete record 30

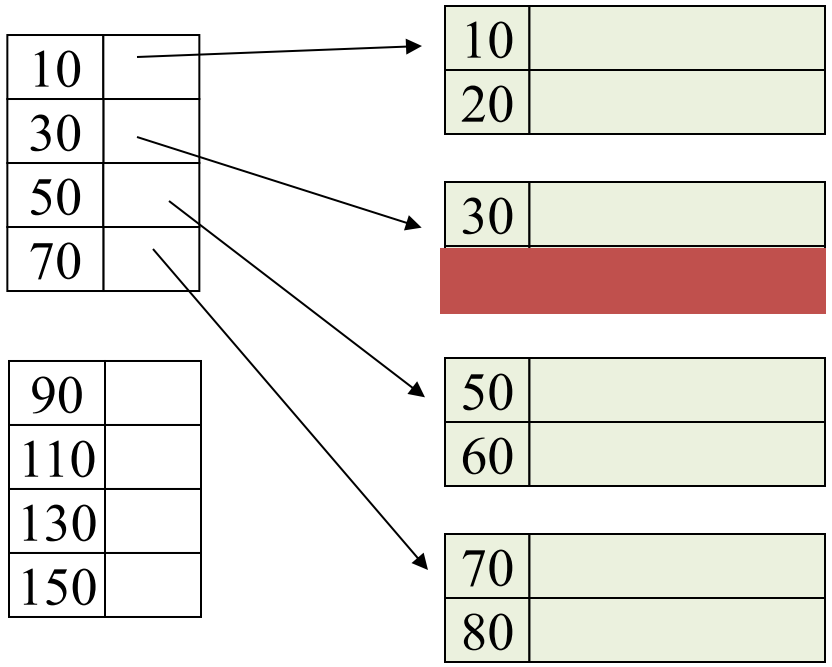
- Same ideas and mechanisms
- Dense indexes may trigger more updates in the index
- Record 40 may or may not move in its data blocks
- Index cannot have free slots in the middle



Sparse Index: Deletion

– Delete record 40

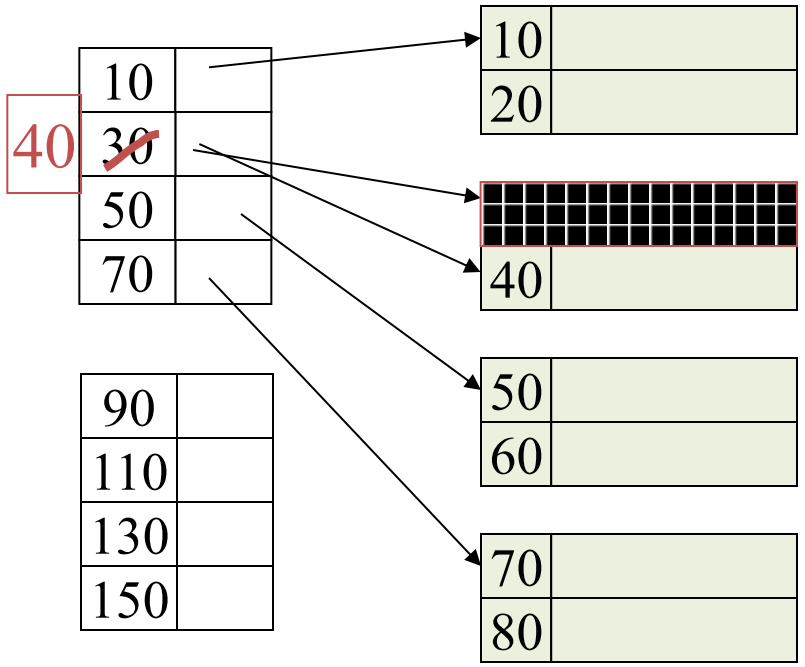
- Index requires no re-organization
- Data block will have some empty space
  - Good to have



Sparse Index: Deletion

– (Instead) Delete record 30

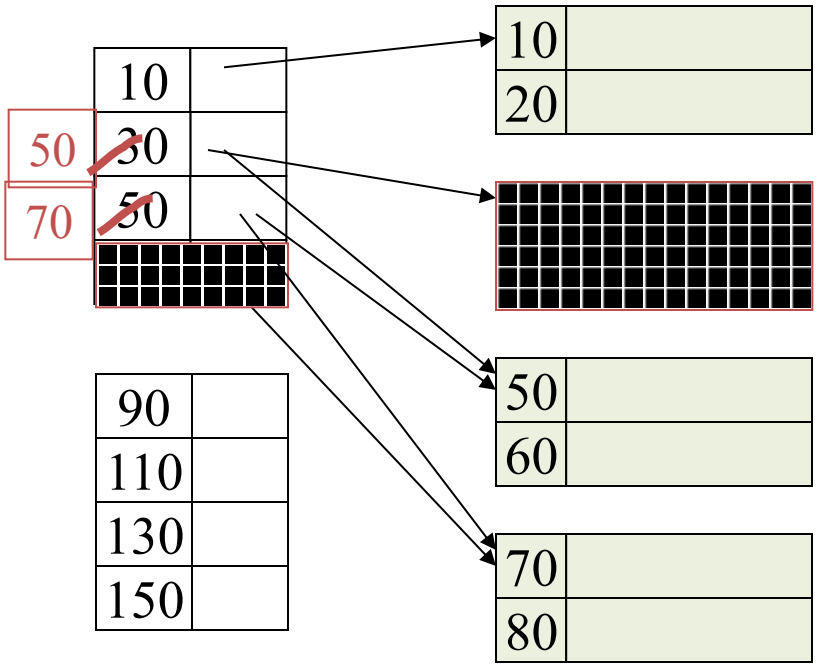
- The value 30 in the index will change
- Record 40 may or may not move



Sparse Index: Deletion

– Delete records 30 & 40

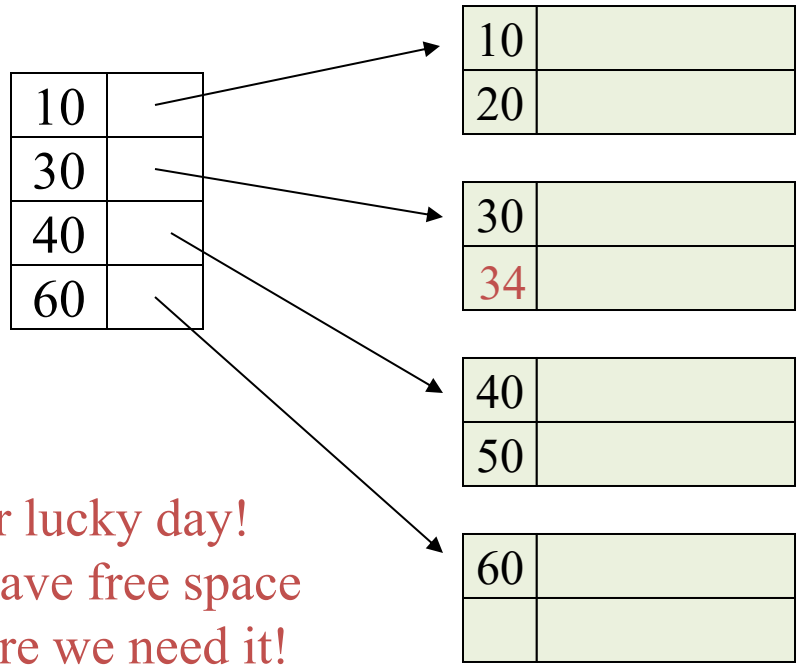
- In the data file, Block 2 will be deleted
- In the index file, do not create empty spaces in the middle
  - Can have empty spaces at the end
  - *Remember: all index entries of the same size*



Sparse Index: Insertion

– Insert record 34

- Good to have free space in each data block
  - Especially if the file is ordered
- DBMSs may keep x%, e.g., 10%, free to make insertions easier



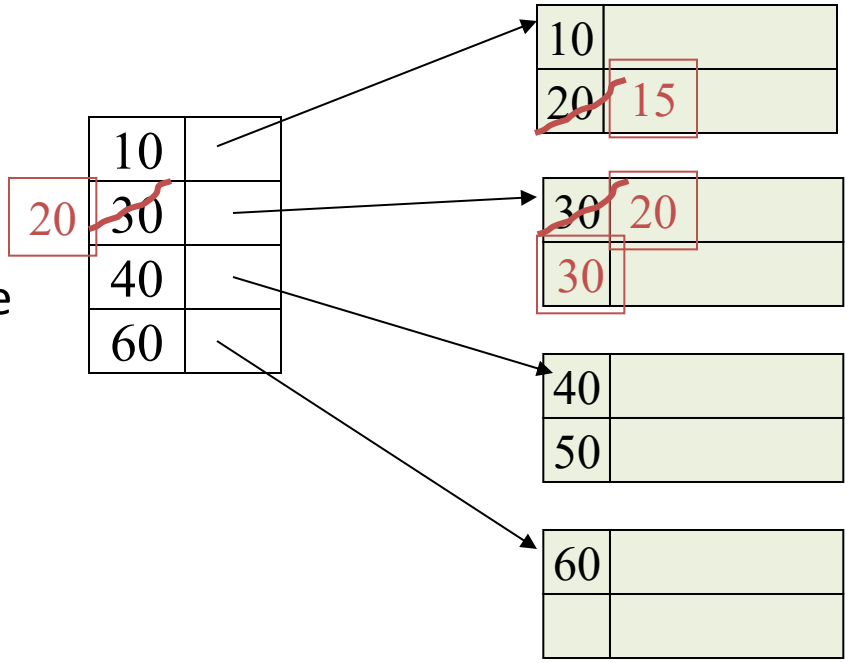
• our lucky day!
we have free space
where we need it!

Sparse Index: Insertion

– Insert record 15

Approach 1 (Immediate Organization)

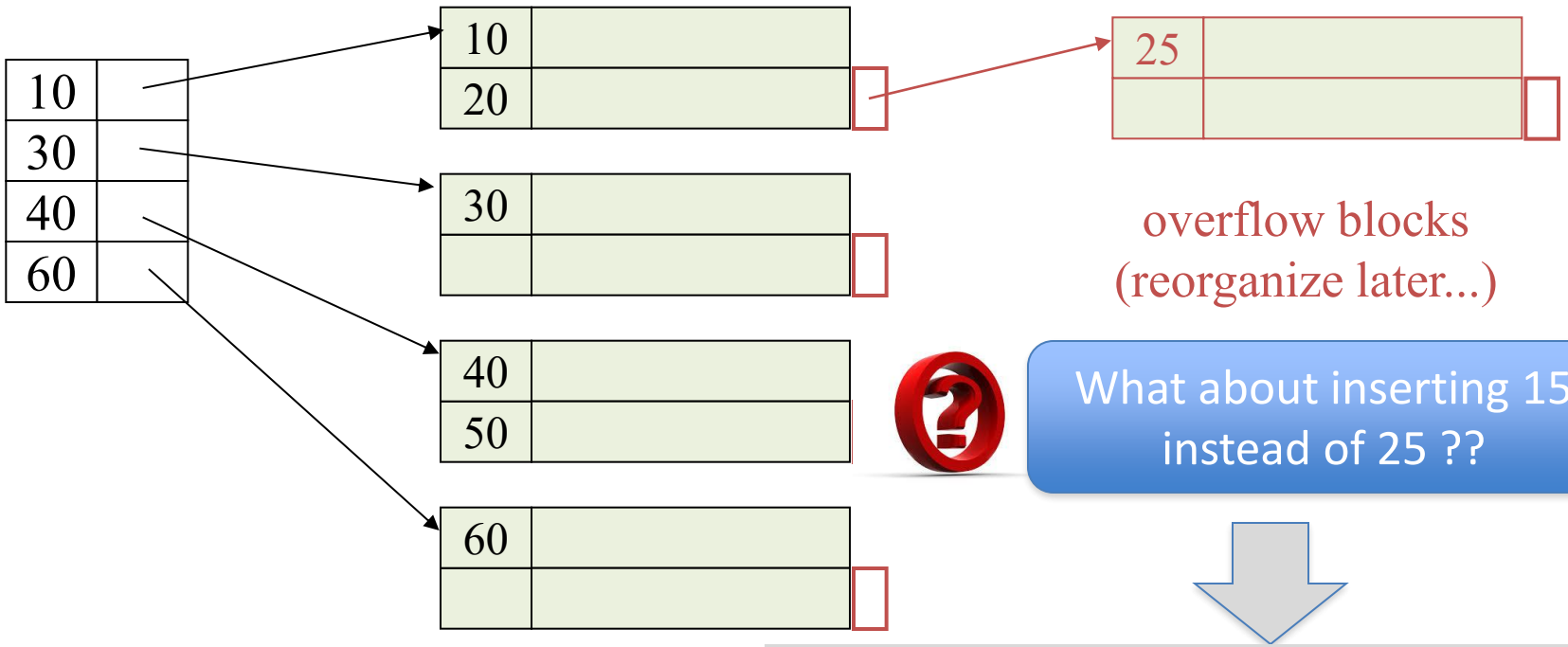
- Move the data records within a block or across blocks to make space for the new record



Other Cheaper Variations ??

Use Of Overflow Blocks

– Insert record 25



- Record 20 will move the overflow bucket
- Still index will not change

Insertion: dense index case

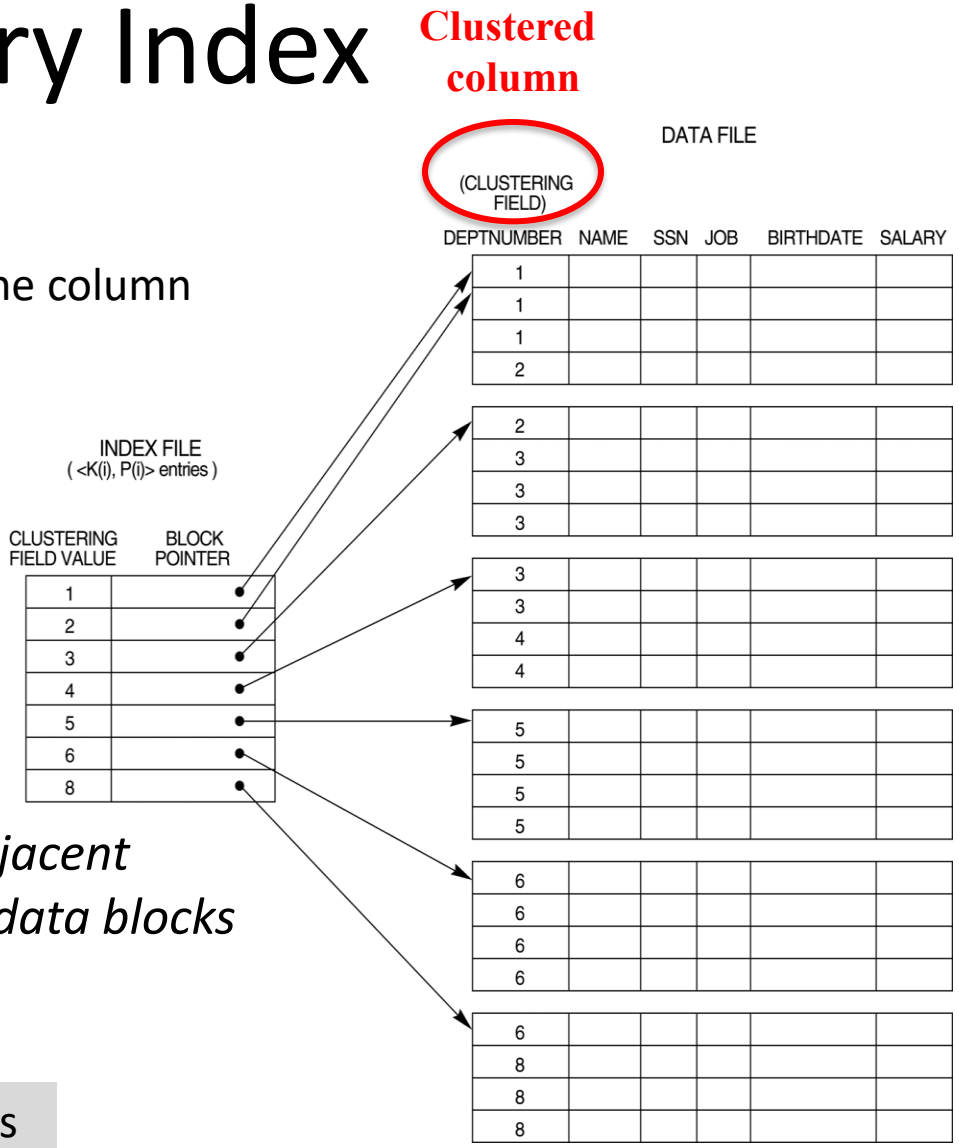
- Similar
- Often more expensive . . .

Index Types

- Dense vs. Sparse
- Primary vs. Secondary 
- One-Level vs. Multi-Level

Primary Index

- **Primary Index is:**
  - An index on the ordering column (the column on which the data file is sorted)
  - Can be dense or sparse
  - Can be one-level or multi-level



Big Advantage

- *Records having the same key (or adjacent keys) are in the same (or adjacent) data blocks*

Leads to sequential I/Os

Back to Bigger Picture

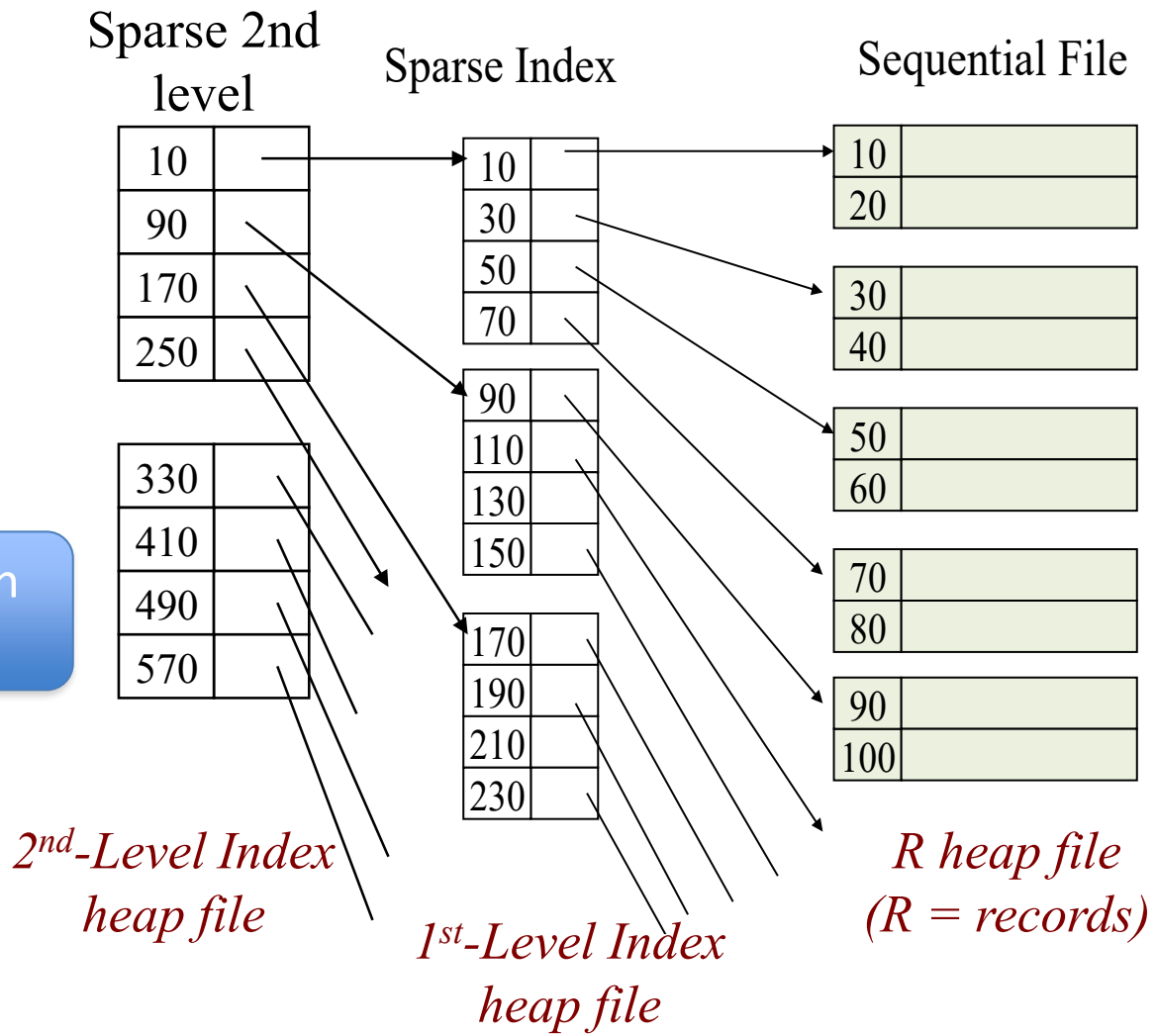
SQL Query

Select ID, name, address
From R
Where ID = 100;

Assume an index is built on column ID

What if I want to search based on Name?

Select ID, name, address
From R
Where name = “Roee”;



Un-Ordered Files & Secondary Indexes

File where records are not ordered on the indexed column

30	
50	

20	
70	

80	
40	

100	
10	

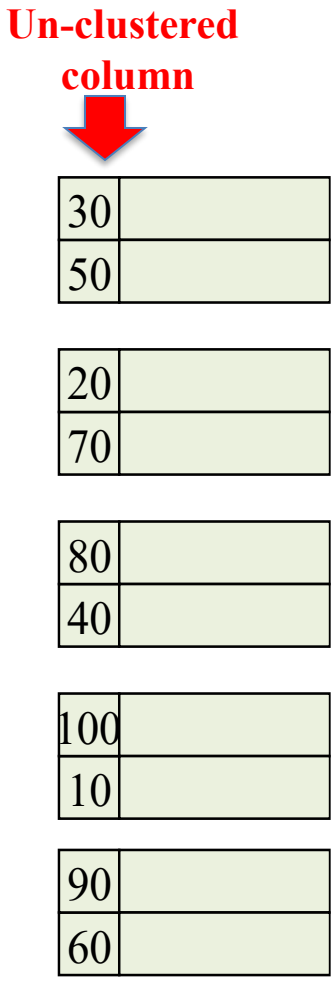
90	
60	

Secondary Indexes

Can we build a sparse index on un-ordered column??

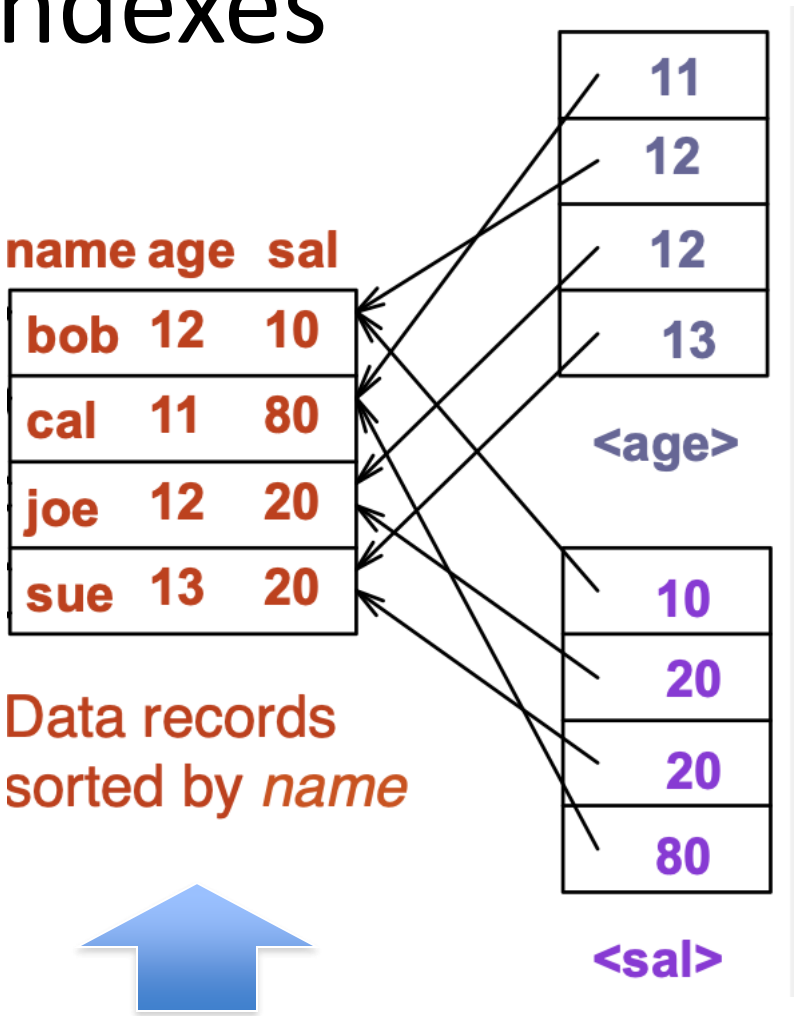
No. We must have an index entry for each data record.

- The file may be ordered on another column, say ***Name***.
  - An index on the ***Name*** column is primary index (Can be sparse or dense)
  - An index on any other column, say ***ID***, is called ***secondary index*** (has to be dense)

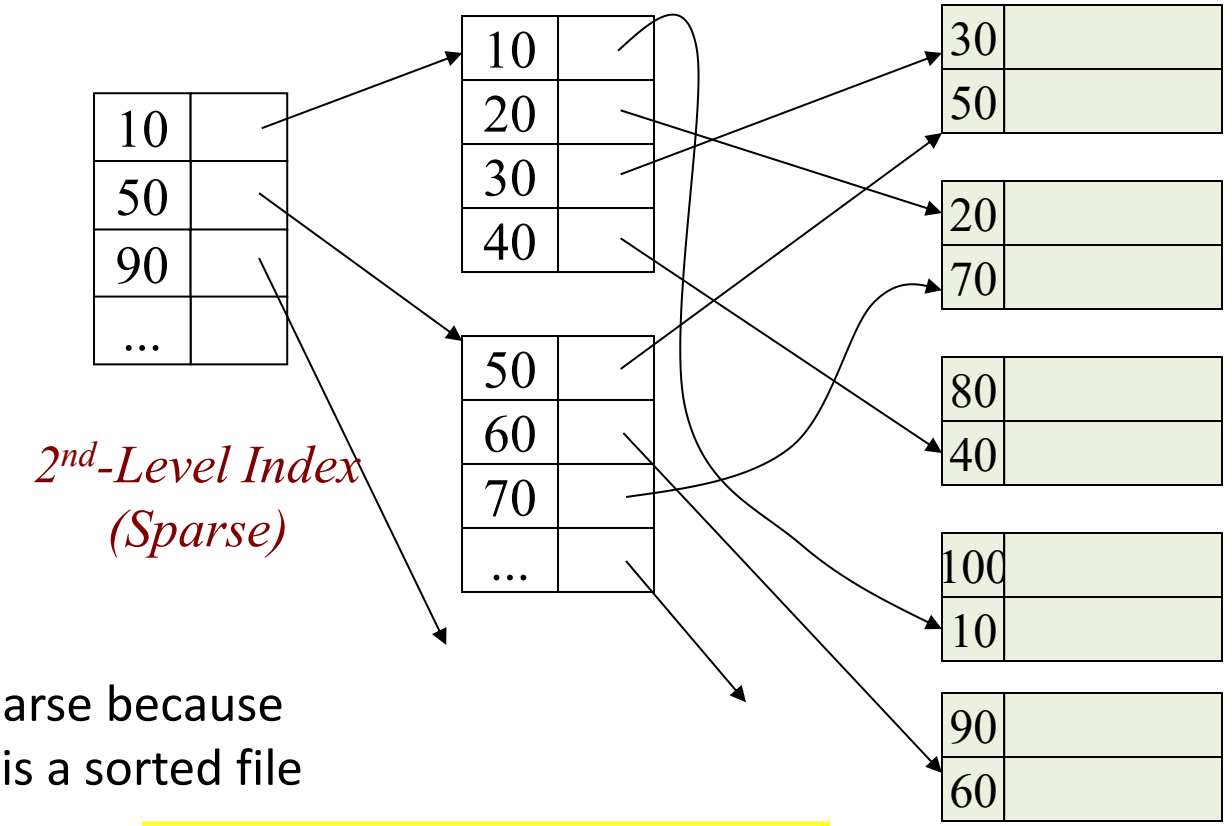


Secondary Indexes

- An index entry for each data record
- Pointers will cause random I/Os (even for same or adjacent values)



Multi-Level Secondary Indexes



- 2nd level can be sparse because the 1st level index is a sorted file

- Lowest level is dense
- Other levels are sparse

Summary

- Index on ordered column → Primary Index
- Index on unordered column → Secondary Index
- Primary Index → Can be dense or sparse (1st level)
- Secondary Index → Can be only dense (1st level)
- Primary Index → Lead to sequential I/Os
- Secondary Index → Lead to random I/Os
- Any additional levels (2nd, 3rd, …) → must be sparse
- Deletion → Data blocks may have empty spaces in the middle
  → Index blocks should not have empty spaces in the middle
- Insertion → Data blocks be added to empty spaces in the middle
  → Index blocks may need to be updated
- Overflow blocks are invisible to the index (do not affect the index entries)